



ENSTA CAMPUS DE BREST

RAPPORT DE PROJET INFORMATIQUE

Application de Gestion de Restaurant : **Le Brestoïs**

DÉVELOPPEURS

Owen-Eagle **Biboka**

Marine **Chabée**

Sommaire

1. Description générale du problème	2
1.1 Introduction et objectifs généraux	2
1.2 Discussion sur les différentes pistes envisagées	2
1.3 Hypothèses réductrices choisies	3
2. Description de l'application proprement dite	3
2.1 Présentation générale et architecture du programme	3
2.2 Diagramme de classes condensé	4
2.3 Mécanismes fondamentaux, héritage et polymorphisme	5
2.4 Guide utilisateur : comment lancer le programme	5
3. Conclusion et perspectives	6
3.1 Stratégie de tests effectués	6
3.2 Limitations actuelles du système	6
3.3 Apports personnels par rapport au sujet initial	7
3.4 Perspectives d'améliorations futures	7

1. DESCRIPTION GÉNÉRALE DU PROBLÈME

1.1 Introduction et objectifs généraux

Dans un restaurant, la communication entre la salle et la cuisine doit être super fluide. Quand un serveur prend une commande, les cuisiniers doivent la recevoir instantanément, sans risque d'erreur ou d'oubli. Les pertes de tickets ou les malentendus ralentissent le service et finissent par agacer les clients. Notre objectif avec l'application "**Le Brestois**" est de régler définitivement ce problème en automatisant tout le flux d'informations du restaurant.

Pour y arriver, on a développé un logiciel complet. D'un côté, les serveurs gèrent les tables et envoient les commandes directement depuis leur interface. De l'autre côté, les cuisiniers suivent la file d'attente des plats en temps réel et mettent à jour l'avancement des préparations. Pour couronner le tout, le gérant a accès à une page de statistiques pour analyser l'activité de l'établissement, voir le chiffre d'affaires du jour, le panier moyen et les plats qui marchent le mieux.

1.2 Discussion sur les différentes pistes envisagées

Avant d'arriver à ce résultat, on a réfléchi à plusieurs façons de structurer notre code :

- **Piste 1 : Un code monolithique tout-en-un.** On aurait pu tout mettre dans un seul et unique fichier géant (l'affichage, la logique des commandes, le calcul des prix). C'était l'option la plus rapide à mettre en place au début, mais on l'a vite rejetée. C'était impossible de travailler proprement à deux là-dessus, et le moindre changement risquait de tout casser.
- **Piste 2 : Une architecture MVC classique avec interrogation régulière.** L'idée était de séparer les vues et les modèles, mais l'interface graphique aurait dû demander au modèle toutes les secondes : "Est-ce qu'il y a une nouvelle commande ?". On a écarté cette piste parce qu'elle fait tourner l'ordinateur dans le vide pour rien.

- **Piste 3 : L'architecture MVC couplée au Pattern Observer.** C'est la solution qu'on a choisie. La logique métier (codée par Marine) et l'interface visuelle (codée par Owen) sont indépendantes. Dès qu'une commande est validée en salle, le modèle se met à jour et envoie un signal asynchrone à la cuisine. C'est propre, moderne et très performant.

1.3 Hypothèses réductrices choisies

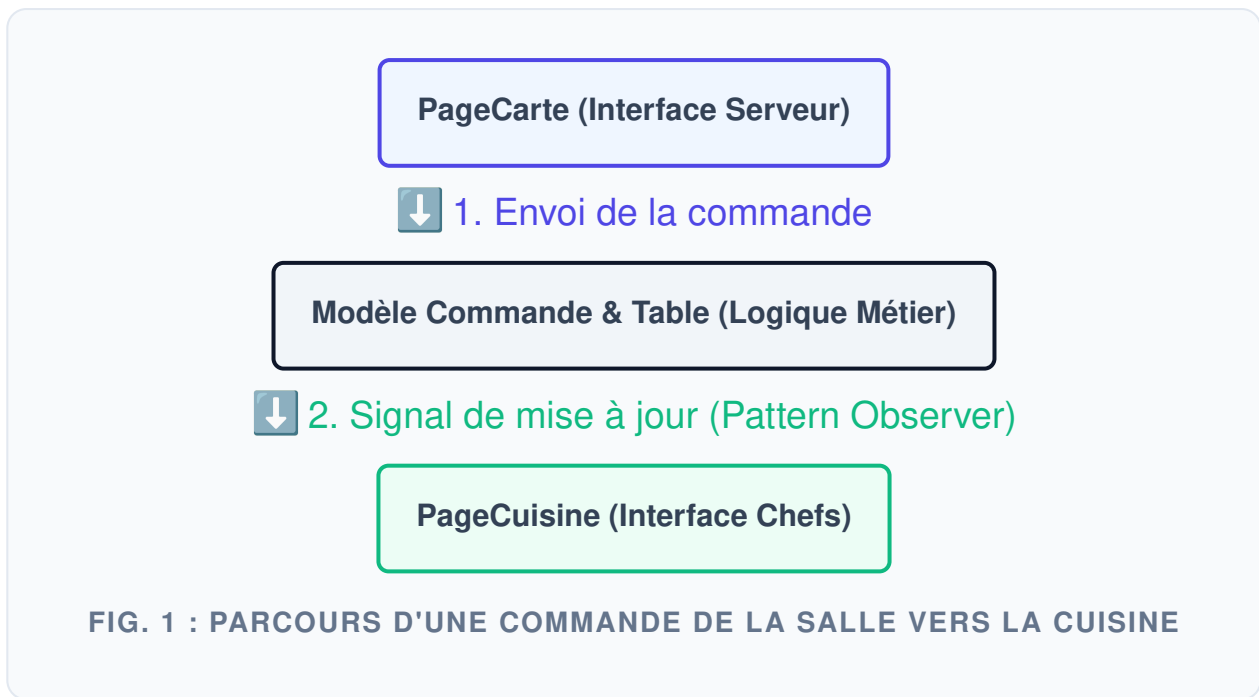
Pour sortir une application stable et fonctionnelle sans y passer des mois, on a dû simplifier le problème avec quelques hypothèses de départ :

1. **Stockage uniquement en mémoire vive** : L'application ne sauvegarde pas encore les données sur un disque dur ou une base de données. Si on ferme le logiciel, la file d'attente et les statistiques du service se réinitialisent à zéro.
2. **Plan de salle fixe** : On est partis sur une configuration standard et bloquée à 10 tables pour le restaurant, numérotées tout simplement de 1 à 10.
3. **Écran unique centralisé** : On suppose que le restaurant utilise un ordinateur principal pour tout centraliser. Le programme tourne en local sur une seule machine et ne gère pas encore de réseau sans fil entre plusieurs tablettes.

2. DESCRIPTION DE L'APPLICATION PROPREMENT DITE

2.1 Présentation générale et architecture du programme

La structure globale du logiciel repose sur une fenêtre maîtresse (`MainWindow`) qui pilote un système d'onglets et de pages interchangeables (`QStackedWidget`). Le schéma suivant montre le parcours d'une information quand un serveur clique sur le bouton de validation depuis la salle.



2.2 Diagramme de classes condensé

Pour comprendre comment nos fichiers collaborent, voici le diagramme UML simplifié de l'application. On y voit clairement comment la vue et les modèles s'articulent grâce aux observateurs.



FIG. 2 : ARCHITECTURE DES CLASSES DE L'APPLICATION

2.3 Mécanismes fondamentaux, héritage et polymorphisme

On a profité à fond des concepts de la Programmation Orientée Objet (POO) pour avoir un code propre et facile à faire évoluer :

- **L'Héritage** : La classe mère `ArticleCarte` regroupe toutes les variables partagées (id, nom, description, prix, disponibilité). Les classes filles comme `Entree` ou `Boisson` en héritent directement, ce qui évite d'écrire dix fois les mêmes propriétés.
- **Le Polymorphisme** : Chaque plat ou boisson gère ses spécificités à sa manière. Par exemple, la méthode `get_temps_preparation()` renvoie le vrai temps estimé pour un

plat cuisiné, mais elle renvoie d'office `0` pour une boisson. L'interface graphique d'Owen peut ainsi manipuler n'importe quel article de la même façon, sans se soucier de savoir s'il s'agit d'un café ou d'un canard.

- **Le Pattern Observer** : C'est la clé de voûte de notre projet. La classe `Commande` hérite d'organisme `Observable`. Dès qu'un statut change, elle appelle sa méthode `notifier()`. L'`ObservateurCuisine` attrape ce signal au vol et met à jour instantanément l'affichage des chefs.

2.4 Guide utilisateur : comment lancer le programme

Pour exécuter le projet sur votre ordinateur, c'est très simple. Voici la marche à suivre pas à pas :

Prérequis

Vérifiez que vous disposez bien de Python 3.8 (ou une version plus récente) sur votre machine, ainsi que de l'outil classique `pip`.

Installation des paquets

Ouvrez votre console ou votre terminal dans le dossier racine du projet, et lancez la commande suivante pour installer les bibliothèques d'interface :

```
pip install PyQt5 qtawesome weasyprint
```

Lancement de l'application

Une fois les téléchargements terminés, il ne reste plus qu'à démarrer le fichier principal :

```
python main.py
```

Le logiciel s'ouvre alors sur la **Page d'Accueil**. Vous pouvez aller sur l'onglet **Carte** pour composer une commande, choisir une table et l'envoyer. Basculez ensuite sur la page

Cuisine en entrant le code secret `marinexowen` pour valider la préparation des plats, puis observez l'impact direct sur l'onglet **Stats** !

3. CONCLUSION ET PERSPECTIVES

3.1 Stratégie de tests effectués

Pour s'assurer que notre application ne plante pas en plein milieu d'un service, on a mis en place plusieurs types de tests :

- **Validation des transitions d'états** : On a poussé le modèle dans ses retranchements en essayant de forcer des parcours de commandes interdits (par exemple, passer un plat directement de "En attente" à "Servi" sans passer par la case cuisine). Le code métier de Marine bloque bien ces actions en levant une `ValueError`, qu'Owen intercepte pour afficher une alerte propre à l'écran.
- **Calculs des indicateurs en direct** : On a simulé des vagues de commandes pour valider les calculs de la page statistiques. Le chiffre d'affaires total, le panier moyen et les jauges de popularité se mettent à jour parfaitement et sans aucun décalage dès qu'on ouvre la page.
- **Gestion des conflits de table** : On a testé le comportement du logiciel lorsqu'un serveur change de table à la volée. Le système gère les abonnements d'observateurs sans créer de doublons, évitant ainsi de dupliquer des lignes de commandes par erreur en cuisine.

3.2 Limitations actuelles du système

Même si le prototype tourne parfaitement, il reste soumis à quelques contraintes techniques :

- **Données volatiles** : Comme tout est gardé en mémoire vive, l'historique s'efface dès qu'on ferme la fenêtre. On ne peut pas suivre l'évolution du restaurant sur plusieurs jours ou plusieurs mois.

- **Utilisation en local** : L'application est faite pour tourner sur un seul écran d'ordinateur. On ne peut pas connecter plusieurs tablettes de serveurs en WiFi à un écran central en cuisine pour le moment.

3.3 Apports personnels par rapport au sujet initial

On a eu envie d'aller un peu plus loin que les demandes de base en ajoutant des options réalistes au projet :

- **Mode serveur sécurisé** : L'accès aux commandes en cuisine est verrouillé par un mot de passe global (`marinexowen`). Ça évite qu'un client ou un serveur farceur ne vienne perturber le travail des chefs depuis l'écran de la salle.
- **Affichage Haute Définition (HighDPI)** : On a configuré PyQt5 pour qu'il gère nativement la densité des écrans modernes. Les polices de caractères et les icônes de la carte restent parfaitement nettes, même sur des affichages 4K ou Retina.
- **Alertes allergènes automatiques** : L'interface graphique d'Owen scanne la liste des ingrédients des plats de Marine. Si un ingrédient sensible est détecté, un badge visuel rouge s'affiche à côté du plat sur la carte pour prévenir instantanément le serveur.

3.4 Perspectives d'améliorations futures

Si on devait pousser **Le Brestois** à un niveau supérieur pour l'utiliser dans un vrai restaurant, on travaillerait sur deux grands axes :

1. **Passage sur une base de données SQLite** : L'idée est de remplacer notre stockage temporaire en mémoire par des tables de données solides. Cela permettrait de sauvegarder définitivement la carte, l'historique des commandes et d'afficher des bilans comptables sur l'année complète.
2. **Adaptation sur tablettes** : Actuellement, le programme tourne sur un ordinateur fixe. L'objectif serait de rendre l'interface compatible et facile à utiliser sur des tablettes tactiles. De cette façon, les serveurs pourraient prendre les commandes directement à la table des clients, ce qui ferait gagner encore plus de temps.